

TP: Introduction à l'analyse statistique bivariée sous Python

Objet

L'objet de ce TP est de mettre en oeuvre l'application de quelques notions de statistique descriptive pour l'analyse de données à l'aide de Python.

Rappel de quelques commandes Python pour la statistiques

Python est plutôt connu pour ses librairies de machine learning et de deep learning dans le domaine de la valorisation de la data (data science, data mining, ...). Il dispose également des librairies statistiques de qualité, notamment avec les références incontournables `scipy` et `statsmodels`.

Quelques packages nécessaires

- `ipython` (travail interactif)
- `numpy` (vecteurs et tableaux)
- `scipy` (algorithmes scientifiques de base, y compris statistiques)
- `matplotlib` (graphiques et visualisation)
- `patsy` (formules statistiques)
- `pandas` (permet de manipuler des Data Frames - Tableaux de Données)
- `statsmodels` (modélisation statistique avancée)
- `seaborn` (visualisation de données statistiques)
- `scikit.learn` (machine learning)
- `xlrd` (lire et écrire des fichiers Microsoft Excel)
- `PyMC` (Statistique bayésienne, y compris MCMC)
- `rpy2` (permet d'utiliser des fonctions R en Python)
- `scipy.stats` (contient la plupart des algorithmes usuels).

Commandes usuelles de statistique

Chargement et lecture de données

• On peut entrer les données manuellement par la création de tableau (arrays)
`array = np.array([1, 2, 3, 4, 5])`
`matrix = np.zeros((3, 3))`

- Ou à partir d'un fichier excel. Il y a deux façons de lire un fichier Microsoft Excel sous pandas : la fonction `read_excel` et la classe `ExcelFile`.
- `read_excel` permet de lire un fichier avec des arguments spécifiques au fichier (par exemple, un même format de données pour les différentes feuilles)
- `ExcelFile` permet de lire un fichier avec des arguments spécifiques aux feuilles (des formats de données différents pour les différentes feuilles)
- Si cela ne fonctionne pas, on peut avoir recours au package Python `xlrd`.
- Ou bien si on dispose d'un fichier de données "fichier.csv"

```
donnees = pd.read_csv('data/fichier.csv')
```

La variable `donnees` est alors une table de données ou *Dataframe*. On note que : • La commande `donnees.head()` permet de n'afficher que les 5 premiers rangs du tableau. • La commande `donnees.shape` renvoie un couple (n ; p) où n est le nombre de lignes et p le nombre de colonnes. • La commande `donnees.columns` permet d'afficher l'ensemble des colonnes du tableau

Commandes de caracteristiques de centralité et de dispersion

Après avoir chargé un fichier de données (csv ou excel....) on définit une table de donnée : par exemple

```
table=donnees[['colone i', 'colone j ', 'colone k']]
```

- la renommer :

```
table=table.rename(columns ={'colone i': 'nv nomination i', 'colone k': 'nv nomination k'})
```

On manipule des tables de données, il faut donc utiliser des commandes propres au module pandas :

- `table.count()` renvoie le nombre de valeurs non-null par colonne
- `table.describe()` renvoie les statistiques de base pour les colonnes numériques
- `table.describe(include=['colone i'])` renvoie les statistiques pour la 'colone i'
- `table.mean()` renvoie la liste des moyennes empiriques (Mean Value en anglais), pour chaque colonne numérique
- `table['nom.de.la.colonne'].mean()` renvoie la moyenne des valeurs d'une colonne précise.
- `table.std()` renvoie la liste des écarts-types pour chaque colonne numérique
- `table['nom.de.la.colonne'].std()` renvoie l'écart-type des valeurs d'une colonne précise.
- `table.median()` renvoie la liste des médianes pour chaque colonne numérique
- `table['nom.de.la.colonne'].median()` renvoie la médiane des valeurs d'une colonne précise.
- `table.min()` renvoie la liste des valeurs minimales pour chaque colonne numérique
- `table['nom.de.la.colonne'].min()` renvoie la valeur minimale d'une colonne précise.
- idem avec `table.max()`
- `table.quantile([0.25, 0.75])` renvoie les quartiles spécifiques Q1 et Q3
- `table.boxplot()` pour les boîtes à moustache
- `table.boxplot(column = "colone i")` pour la boîte à moustache de la colone i
- `table.var` pour la variance
- `table.corr()` pour la matrice de corrélation
- `table['colone i'].corr(df['colone j'])` pour la corrélation entre deux colone i et j

1 Manipulation

- 1- Imporation des modules :

```
import os # pour gérer les répertoires
import numpy as np
import scipy.stats as sps # pour les fonctions statistiques
import pandas
```

- 2- Chargement et lecture des données : `D1 = pd.read_csv('data/stats_data_TP.csv')`

```
table1=D1[['An', 'taux fert ', 'esp vie']]
```

```
table1=table1.rename(columns ={'taux fert': 'Taux de Fertilité', 'esp vie': 'Espérance
```

- ```

de vie'}})
table1 = table1.dropna() #supprime les lignes avec des données manquantes

```
- Question-1 Renvoyer à l'aide des commande appropriées : le mode, la mediane , la moyenne empirique et la boîte à moustache pour la variable : 'Taux de fertilité' puis pour la variable : 'Espérance de vie'?
- 4 Affichage des données : (nuage de points)
- ```

X = table1['Espérance de vie']
Y = table1['Taux de Fertilité']
plt.grid()
plt.plot(X,Y,'+')
plt.show()

```
- Question-2 Comment afficher également le point moyen du nuage $(\overline{X}, \overline{Y})$?
- 4 Corrélation linéaire : Question-2 a l'aide des commandes python calculer la covariance de X et Y notée cov(x,y) puis le coefficient de corrélation r? interpréter la relation entre X et Y selon la valeur de r?
- 5 Ecrire un code python permettant de représenter la droite de régression linéaire de Y en X au dessus du nuage de points (X; Y)?